

MATERIA • INF560 — DESARROLLO WEB BACKEND

UNIDAD 4

Desarrollo de API y Servicios Web

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

4.1 Fundamentos de API REST

- 4.1.1 Métodos HTTP aplicados a REST
- 4.1.2 Recursos y endpoints
- 4.1.3 Códigos de estado en API REST

4.2 Diseño de API RESTful

- 4.2.1 Mejores prácticas y convenciones
- 4.2.2 Versionado de API
- 4.2.3 Manejo de errores y respuestas estandarizadas

4.3 Introducción a GraphQL

- 4.3.1 Conceptos básicos
- 4.3.2 Diferencias con REST

4.4 Autenticación y autorización en API

- 4.4.1 JSON Web Tokens (JWT)
- 4.4.2 OAuth2

4.5 Documentación de API

- 4.5.1 Herramientas de documentación: Swagger / OpenAPI

COMPETENCIA DE LA UNIDAD

Diseña y construye interfaces de programación de aplicaciones (API) orientadas a servicios web, aplicando los principios arquitectónicos REST, los métodos del protocolo HTTP y las convenciones de diseño RESTful para exponer recursos de forma coherente y predecible. Implementa mecanismos de autenticación y autorización basados en tokens, contrasta el enfoque REST con GraphQL, y documenta los contratos de servicio mediante la especificación OpenAPI, valorando la seguridad, la mantenibilidad y la experiencia del consumidor de la API.

Índice de Contenidos

4.1 Fundamentos de API REST	3
4.1.1 Métodos HTTP aplicados a REST	4
4.1.2 Recursos y endpoints	5
4.1.3 Códigos de estado en API REST	5
4.2 Diseño de API RESTful.....	7
4.2.1 Mejores prácticas y convenciones	7
4.2.2 Versionado de API	7
4.2.3 Manejo de errores y respuestas estandarizadas	8
4.3 Introducción a GraphQL	10
4.3.1 Conceptos básicos	10
4.3.2 Diferencias con REST	11
4.4 Autenticación y autorización en API.....	12
4.4.1 JSON Web Tokens (JWT)	12
4.4.2 OAuth2	13
4.5 Documentación de API.....	14
4.5.1 Herramientas de documentación: Swagger / OpenAPI.....	14
Síntesis de la unidad.....	15

Unidad 4: Desarrollo de API y Servicios Web

En las unidades anteriores construimos aplicaciones donde el servidor generaba directamente las vistas HTML que consumía el navegador. En esta unidad damos un paso decisivo: separamos el **backend** como un proveedor de datos y servicios independiente de la interfaz que los consume. El servidor deja de “dibujar páginas” y pasa a **exponer recursos** a través de una **API (Application Programming Interface)**, de modo que aplicaciones web, móviles, de escritorio o incluso otros servidores puedan integrarse con la misma lógica de negocio. Estudiaremos cómo diseñar estas interfaces siguiendo el estilo arquitectónico REST, cómo protegerlas, una alternativa moderna llamada GraphQL y cómo documentarlas profesionalmente.

4.1 Fundamentos de API REST

Una **API web** es un conjunto de reglas y puntos de acceso que permite que dos sistemas de software se comuniquen a través de la red, normalmente sobre el protocolo HTTP. **REST** (*Representational State Transfer*) no es un protocolo ni un estándar, sino un **estilo arquitectónico** propuesto por Roy Fielding en su tesis doctoral del año 2000. Define un conjunto de restricciones que, al respetarse, producen sistemas distribuidos escalables, simples y desacoplados. Cuando una API cumple esas restricciones se la denomina **RESTful**.

RECURSO

Un **recurso** es cualquier entidad de información que la API expone y que puede ser nombrada y direccionada: un usuario, un libro, una factura, una colección de productos. Cada recurso se identifica mediante una **URI** única, y el cliente interactúa con **representaciones** de ese recurso (habitualmente en formato JSON).

Las seis restricciones que caracterizan a REST son las siguientes:

Restricción	Descripción
Cliente–Servidor	Separa la interfaz de usuario del almacenamiento de datos, mejorando la portabilidad y permitiendo que ambos evolucionen de forma independiente.
Sin estado (stateless)	Cada petición debe contener toda la información necesaria para ser atendida. El servidor no guarda estado de sesión entre peticiones.
Cacheable	Las respuestas deben indicar si pueden almacenarse en caché, reduciendo la carga y mejorando el rendimiento percibido.
Interfaz uniforme	Conjunto coherente de reglas (identificación de recursos, manipulación mediante representaciones, mensajes autodescriptivos, HATEOAS).
Sistema en capas	El cliente no sabe si se conecta al servidor final o a un intermediario (proxy, balanceador, gateway).
Código bajo demanda (opcional)	El servidor puede enviar código ejecutable (por ejemplo, JavaScript) para extender al cliente. Es la única restricción opcional.

STATELESS EN LA PRÁCTICA

Que REST sea **sin estado** implica que la autenticación no se guarda en una sesión del servidor: el cliente envía sus credenciales (normalmente un **token**) en cada petición. Esto facilita el escalado horizontal, porque cualquier instancia del servidor puede atender cualquier petición sin compartir memoria de sesión.

4.1.1 Métodos HTTP aplicados a REST

REST aprovecha los **verbos del protocolo HTTP** para expresar la **acción** que se quiere realizar sobre un recurso. Así, la misma URI puede comportarse de forma distinta según el método empleado. Esta correspondencia entre verbos HTTP y operaciones se conoce informalmente como mapeo **CRUD** (*Create, Read, Update, Delete*).

Método	Operación CRUD	Propósito	Seguro	Idempotente
GET	Read	Recupera la representación de un recurso o colección.	Sí	Sí
POST	Create	Crea un nuevo recurso dentro de una colección.	No	No
PUT	Update	Reemplaza por completo un recurso existente.	No	Sí
PATCH	Update	Modifica parcialmente un recurso existente.	No	No*
DELETE	Delete	Elimina un recurso.	No	Sí

SEGURIDAD E IDEMPOTENCIA

Método seguro: no modifica el estado del servidor; solo lee información (*GET, HEAD, OPTIONS*).

Método idempotente: ejecutarlo una o varias veces produce el mismo efecto en el servidor. *PUT* y *DELETE* lo son; *POST* no, porque cada llamada crea un recurso nuevo. (**PATCH* puede ser idempotente o no, según cómo se defina la operación).

El siguiente ejemplo muestra el ciclo de vida completo de un recurso *libro* mediante los distintos métodos:

```
# Crear un libro nuevo (la respuesta incluye el id asignado)
POST /api/libros
Content-Type: application/json

{ "titulo": "Clean Code", "autor_id": 7, "anio": 2008 }

# Obtener la lista completa de libros
GET /api/libros

# Obtener un libro concreto
GET /api/libros/42

# Reemplazar TODOS los campos del libro 42
PUT /api/libros/42
{ "titulo": "Clean Code", "autor_id": 7, "anio": 2009 }

# Modificar SOLO el año del libro 42
PATCH /api/libros/42
{ "anio": 2009 }

# Eliminar el libro 42
DELETE /api/libros/42
```

En Laravel, este mapeo se declara de forma compacta con rutas de recurso de API. Una sola línea genera las cinco rutas canónicas asociadas a un controlador:

```
// routes/api.php
use App\Http\Controllers\LibroController;

Route::apiResource('libros', LibroController::class);

// Equivale a:
// GET      /api/libros           -> index()
// POST     /api/libros           -> store()
// GET      /api/libros/{libro}   -> show()
// PUT      /api/libros/{libro}   -> update() (también PATCH)
// DELETE   /api/libros/{libro}   -> destroy()
```

4.1.2 Recursos y endpoints

Un **endpoint** es la combinación de una **URI** y un **método HTTP** que el cliente invoca para operar sobre un recurso. El diseño de estas URI es una de las decisiones más visibles de una API: una nomenclatura clara y coherente hace que la API sea intuitiva y predecible.

Principios fundamentales para nombrar recursos:

- Usar **sustantivos, no verbos**. La acción ya la indica el método HTTP. Se escribe `GET /api/libros`, nunca `GET /api/obtenerLibros`.
- Usar **plural** para las colecciones: `/api/usuarios`, `/api/pedidos`.
- Reflejar la **jerarquía** mediante anidamiento: `/api/autores/7/libros` representa los libros del autor 7.
- Usar **minúsculas** y separar palabras con guiones: `/api/ordenes-de-compra`.
- Emplear la **cadena de consulta** (query string) para filtrar, ordenar y paginar, no para identificar recursos: `/api/libros?autor=7&orden=anio&pagina=2`.

Endpoint	Significado
GET <code>/api/libros</code>	Colección: todos los libros.
GET <code>/api/libros/42</code>	Recurso individual: el libro con id 42.
GET <code>/api/autores/7/libros</code>	Subcolección: libros del autor 7.
POST <code>/api/libros</code>	Crea un libro en la colección.
DELETE <code>/api/libros/42</code>	Elimina el libro 42.

EVITA VERBOS EN LAS URI

Endpoints como `/api/crearLibro` o `/api/libros/borrar/42` rompen la interfaz uniforme de REST y multiplican innecesariamente la superficie de la API. Reserva las acciones que no encajan en CRUD (por ejemplo, `/api/pedidos/42/cancelar`) para casos verdaderamente excepcionales, y considéralas como subrecursos de proceso.

4.1.3 Códigos de estado en API REST

Cada respuesta HTTP incluye un **código de estado** de tres dígitos que comunica de forma estandarizada el resultado de la petición. Usarlos correctamente es esencial: el código

permite que el cliente reaccione de manera programática sin tener que interpretar el cuerpo del mensaje. Los códigos se agrupan en cinco familias según su primer dígito.

Familia	Significado general
1xx — Informativo	La petición fue recibida y el proceso continúa.
2xx — Éxito	La petición se recibió, entendió y procesó correctamente.
3xx — Redirección	Se requiere una acción adicional para completar la petición.
4xx — Error del cliente	La petición contiene errores o no puede ser atendida.
5xx — Error del servidor	El servidor falló al procesar una petición aparentemente válida.

Códigos más usados en APIs REST:

Código	Nombre	Cuándo usarlo
200	OK	Éxito genérico en GET, PUT o PATCH; la respuesta incluye datos.
201	Created	Un POST creó un recurso. Conviene devolver la cabecera Location con su URI.
204	No Content	Éxito sin cuerpo de respuesta; típico tras un DELETE.
400	Bad Request	Petición malformada o sintácticamente incorrecta.
401	Unauthorized	Falta autenticación o las credenciales son inválidas.
403	Forbidden	Autenticado, pero sin permiso para el recurso.
404	Not Found	El recurso solicitado no existe.
409	Conflict	Conflicto de estado (p. ej., un valor único duplicado).
422	Unprocessable Entity	La sintaxis es correcta pero la validación de negocio falla.
500	Internal Server Error	Error inesperado del servidor.

DISTINCIÓN 401 VS 403

Aunque suelen confundirse, **401 Unauthorized** significa “no sé quién eres” (falta o falla la autenticación), mientras que **403 Forbidden** significa “sé quién eres, pero no puedes hacer esto” (falla la autorización). De igual modo, **422** se prefiere a **400** cuando el JSON está bien formado pero un campo no cumple las reglas de validación (p. ej., un correo sin formato válido).

En Laravel, el código de estado se especifica como segundo argumento al construir la respuesta JSON:

```
// Creación exitosa de un recurso
return response()->json($libro, 201);

// Recurso no encontrado
return response()->json(['mensaje' => 'Libro no encontrado'], 404);

// Eliminación sin contenido de respuesta
return response()->noContent(); // 204
```

4.2 Diseño de API RESTful

Cumplir las restricciones de REST es el punto de partida; construir una API **excelente** exige además un conjunto de convenciones y decisiones de diseño que la hagan consistente, evolucionable y agradable de consumir. Esta sección reúne las prácticas que la industria ha consolidado.

4.2.1 Mejores prácticas y convenciones

- **Consistencia ante todo.** Mantener un mismo estilo de nombres, formato de fechas (ISO 8601), convención de claves JSON (`snake_case` o `camelCase`) y estructura de respuestas en toda la API.
- **Paginación.** Nunca devolver colecciones ilimitadas. Usar parámetros como `?pagina=2&por_pagina=20` e incluir metadatos de paginación (total, página actual, enlaces).
- **Filtrado, ordenación y búsqueda** mediante la cadena de consulta: `?estado=activo&orden=-fecha&buscar=clean`.
- **Negociación de contenido.** Respetar las cabeceras `Accept` y `Content-Type`; JSON es el formato dominante (`application/json`).
- **HTTPS obligatorio.** Toda API debe servirse sobre TLS para proteger credenciales y datos en tránsito.
- **HATEOAS (cuando aporte valor).** Incluir en las respuestas enlaces a acciones relacionadas, permitiendo que el cliente “navegue” la API sin acoplar las URI en su código.

ENVELOPE DE RESPUESTA

Muchas APIs envuelven los datos en una estructura consistente (un **envelope**) con campos como `data`, `meta` y `links`. Esto da uniformidad y deja espacio para metadatos sin contaminar el recurso. En Laravel, los **API Resources** producen exactamente este patrón de forma automática.

```
{
  "data": [
    { "id": 42, "titulo": "Clean Code", "anio": 2008 }
  ],
  "meta": { "pagina": 1, "por_pagina": 20, "total": 137 },
  "links": { "next": "/api/libros?pagina=2", "prev": null }
}
```

4.2.2 Versionado de API

Una API publicada tiene **consumidores** que dependen de su contrato. Cuando se introducen cambios incompatibles (eliminar un campo, cambiar un tipo, renombrar un endpoint), no se puede romper a esos clientes de un día para otro. El **versionado** permite evolucionar la API publicando nuevas versiones mientras se mantienen las anteriores durante un periodo de transición.

Estrategia	Ejemplo	Ventajas / Inconvenientes
En la URI	<code>/api/v1/libros</code>	La más visible y sencilla de enrutar; la versión queda explícita. Es la más usada.

Estrategia	Ejemplo	Ventajas / Inconvenientes
En cabecera	Accept: application/vnd.api.v1+json	URI “limpias”; más acorde a la pureza REST, pero menos visible y más difícil de probar.
En query string	/api/libros?version=1	Fácil de implementar; ensucia las URL y se mezcla con filtros.

QUÉ ES UN CAMBIO INCOMPATIBLE

Agregar un campo nuevo o un endpoint nuevo suele ser **retrocompatible** y no exige nueva versión. En cambio, **eliminar o renombrar** campos, cambiar tipos de datos, modificar códigos de estado o alterar la semántica de un endpoint **sí** rompe a los clientes y obliga a versionar.

```
// routes/api.php - versionado por URI con grupos de rutas
Route::prefix('v1')->group(function () {
    Route::apiResource('libros', V1\LibroController::class);
});

Route::prefix('v2')->group(function () {
    Route::apiResource('libros', V2\LibroController::class);
});
```

4.2.3 Manejo de errores y respuestas estandarizadas

Una API profesional no solo responde bien al éxito; comunica los **errores** de forma clara, consistente y accionable. El cliente debe poder distinguir programáticamente qué falló y, cuando proceda, mostrar mensajes útiles al usuario final. La combinación adecuada es: un **código de estado HTTP correcto** más un **cuerpo de error estructurado**.

El estándar **RFC 7807 (Problem Details for HTTP APIs)** propone un formato común para los cuerpos de error, con campos como `type`, `title`, `status`, `detail` e `instance`:

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{
  "type": "https://api.ejemplo.com/errores/validacion",
  "title": "Error de validación",
  "status": 422,
  "detail": "El cuerpo de la petición contiene campos inválidos.",
  "errors": {
    "titulo": ["El título es obligatorio."],
    "anio": ["El año debe ser un número entero."]
  }
}
```

Laravel facilita enormemente este patrón. La validación mediante `Form Request` devuelve automáticamente un `422` con un objeto `errors` estructurado cuando la petición espera JSON, y los errores no controlados pueden centralizarse en el manejador de excepciones:

```
// app/Http/Requests/StoreLibroRequest.php
public function rules(): array
{
```



```
return [
    'titulo' => ['required', 'string', 'max:255'],
    'autor_id' => ['required', 'integer', 'exists:autores,id'],
    'anio' => ['required', 'integer', 'min:1450'],
];

// Si la validación falla y el cliente envía 'Accept: application/json',
// Laravel responde 422 con: { "message": ..., "errors": { ... } }
```

REGLA DE ORO DEL MANEJO DE ERRORES

Un buen mensaje de error responde a tres preguntas: **qué** salió mal (código y título), **por qué** (detalle legible) y, si es posible, **cómo** corregirlo (errores por campo). Nunca se deben filtrar trazas de pila, consultas SQL ni detalles internos en producción.

4.3 Introducción a GraphQL

REST dominó el diseño de APIs durante más de una década, pero presenta limitaciones cuando los clientes necesitan datos muy variables o relacionados. **GraphQL**, creado por Facebook en 2012 y liberado públicamente en 2015, es un **lenguaje de consulta para APIs** y un entorno de ejecución que permite al cliente pedir exactamente los datos que necesita, ni más ni menos, en una única petición.

4.3.1 Conceptos básicos

En GraphQL, el servidor expone un **único endpoint** (habitualmente `/graphql`) y publica un **esquema** (`schema`) fuertemente tipado que describe todos los datos disponibles y las operaciones posibles. El cliente envía **consultas** que reflejan la forma exacta de la respuesta deseada.

Concepto	Descripción
Schema	Contrato tipado que define los tipos de datos, sus campos y las operaciones disponibles.
Type	Estructura de un objeto del dominio (p. ej., Libro, Autor) con sus campos y tipos.
Query	Operación de lectura. Equivale conceptualmente a un GET en REST.
Mutation	Operación de escritura (crear, actualizar, eliminar).
Subscription	Operación en tiempo real: el servidor envía datos al cliente cuando hay cambios.
Resolver	Función del servidor que sabe cómo obtener el valor de un campo concreto.

Un esquema sencillo y una consulta que lo aprovecha se verían así:

```
# --- Definición del esquema (servidor) ---
type Autor {
  id: ID!
  nombre: String!
  libros: [Libro!]!
}

type Libro {
  id: ID!
  titulo: String!
  anio: Int
  autor: Autor!
}

type Query {
  libro(id: ID!): Libro
  libros: [Libro!]!
}
```

```
# --- Consulta (cliente): pide solo lo que necesita ---
query {
  libro(id: 42) {
    titulo
    autor {
```

```

    nombre
  }
}
}

# --- Respuesta: misma forma que la consulta ---
{
  "data": {
    "libro": {
      "titulo": "Clean Code",
      "autor": { "nombre": "Robert C. Martin" }
    }
  }
}

```

UN SOLO VIAJE, DATOS RELACIONADOS

Observa que en una sola petición el cliente obtuvo el título del libro **y** el nombre de su autor. En REST esto habría requerido normalmente dos llamadas (`/libros/42` y luego `/autores/{id}`), o un endpoint especializado.

4.3.2 Diferencias con REST

GraphQL no “reemplaza” a REST: son enfoques con compromisos distintos. Dos problemas clásicos de REST que GraphQL resuelve de origen son el **over-fetching** (recibir más campos de los necesarios) y el **under-fetching** (necesitar varias llamadas para reunir datos relacionados, el llamado problema “N+1 de red”).

Aspecto	REST	GraphQL
Endpoints	Múltiples, uno por recurso.	Uno solo (<code>/graphql</code>).
Método HTTP	GET, POST, PUT, PATCH, DELETE.	Casi siempre POST.
Forma de la respuesta	Fija; la define el servidor.	La define el cliente en la consulta.
Over/under-fetching	Frecuente.	Se evita por diseño.
Tipado	No impuesto por el estilo.	Esquema fuertemente tipado.
Caché HTTP	Sencilla (basada en URL y verbos).	Más compleja; suele requerir caché a nivel de cliente.
Códigos de estado	Ricos y semánticos (2xx, 4xx, 5xx).	Suele responder 200 con errores en el cuerpo.
Curva de aprendizaje	Baja; usa HTTP directamente.	Mayor; requiere esquema, resolvers y herramientas.

¿CUÁNDO ELEGIR CADA UNO?

REST brilla en APIs públicas, operaciones CRUD claras, cuando se quiere aprovechar la caché HTTP y la simplicidad. **GraphQL** brilla cuando hay muchos clientes con necesidades de datos heterogéneas (web, móvil), relaciones complejas y se busca minimizar el número de peticiones. La elección depende del contexto, no de la moda.

4.4 Autenticación y autorización en API

Como las APIs REST son **sin estado**, no pueden depender de sesiones tradicionales del servidor para recordar al usuario. En su lugar, cada petición debe acreditar **quién** la realiza (**autenticación**) y el servidor debe decidir **qué** puede hacer ese usuario (**autorización**). Estos dos conceptos, aunque relacionados, son distintos y no deben confundirse.

AUTENTICACIÓN VS AUTORIZACIÓN

Autenticación (AuthN): verifica la **identidad** — “¿eres realmente quien dices ser?”. Devuelve 401 si falla.

Autorización (AuthZ): verifica los **permisos** — “¿tienes derecho a hacer esto?”. Devuelve 403 si falla.

4.4.1 JSON Web Tokens (JWT)

Un **JWT** es un estándar abierto (RFC 7519) para transmitir información entre partes como un objeto JSON **compacto y firmado digitalmente**. Es el mecanismo más extendido para autenticar APIs sin estado: tras iniciar sesión, el servidor entrega un token que el cliente envía en cada petición, normalmente en la cabecera `Authorization: Bearer <token>`.

Un JWT consta de **tres partes** separadas por puntos (`header.payload.signature`), cada una codificada en Base64URL:

Parte	Contenido
Header (cabecera)	Tipo de token (JWT) y algoritmo de firma (p. ej., HS256, RS256).
Payload (carga)	Los claims: datos del usuario (sub, name) y metadatos (iat, exp, iss).
Signature (firma)	Hash criptográfico que garantiza que el token no fue alterado.

EL PAYLOAD NO ESTÁ CIFRADO

El contenido de un JWT está **codificado** en Base64URL, no **cifrado**. Cualquiera puede leer el payload. La firma garantiza **integridad** (que nadie lo modificó), no **confidencialidad**. Nunca incluyas contraseñas ni datos sensibles en el payload.

El flujo típico de autenticación con JWT es el siguiente:

1. El cliente envía credenciales (`POST /api/login`) con usuario y contraseña.
2. El servidor las verifica y, si son válidas, genera y firma un JWT que devuelve al cliente.
3. El cliente almacena el token y lo envía en la cabecera `Authorization` de cada petición protegida.
4. El servidor valida la firma y la expiración del token, identifica al usuario y atiende la petición. No consulta ninguna sesión.
5. Cuando el token caduca, el cliente usa un `refresh token` para obtener uno nuevo sin volver a introducir credenciales.

```
# Login: el cliente recibe el token
POST /api/login
{ "email": "ana@uatf.edu.bo", "password": "secreto" }

# Respuesta
{
  "access_token": "eyJhbGciOiJIUzI1Ni..."
```

```

    "token_type": "Bearer",
    "expires_in": 3600
  }

  # Petición protegida posterior
  GET /api/perfil
  Authorization: Bearer eyJhbGciOiJIUzI1NiU...

```

JWT EN LARAVEL

Laravel ofrece **Sanctum** para tokens de API simples (ideal para SPAs y apps móviles propias) y **Passport** para escenarios OAuth2 completos. Para JWT puro existe además el paquete `tymon/jwt-auth`. La elección depende de si necesitas un servidor OAuth2 o solo emitir tokens a tus propios clientes.

4.4.2 OAuth2

Mientras que JWT es un **formato de token**, **OAuth2** (RFC 6749) es un **marco de autorización** que define cómo una aplicación puede obtener acceso limitado a los recursos de un usuario **sin conocer su contraseña**. Es el mecanismo detrás del botón “Iniciar sesión con Google” o “Conectar con GitHub”: el usuario autoriza a una aplicación de terceros a actuar en su nombre, con un alcance (*scope*) acotado.

OAuth2 define **cuatro roles** fundamentales:

Rol	Descripción
Resource Owner	El usuario, dueño de los datos protegidos.
Client	La aplicación que solicita acceso a los datos del usuario.
Authorization Server	Autentica al usuario y emite los tokens de acceso.
Resource Server	La API que aloja los recursos protegidos y valida los tokens.

OAuth2 contempla varios **flujos** (*grant types*) según el tipo de cliente y el nivel de confianza:

Flujo (grant)	Uso recomendado
Authorization Code (con PKCE)	Apps web y móviles con interacción del usuario. Es el flujo recomendado actualmente.
Client Credentials	Comunicación servidor-a-servidor, sin usuario implicado.
Refresh Token	Renovar el token de acceso cuando caduca, sin reautenticar.
Resource Owner Password	Solo para clientes de máxima confianza; desaconsejado en nuevos desarrollos.

JWT Y OAUTH2 TRABAJAN JUNTOS

No son competidores. OAuth2 describe el **proceso** de obtener autorización; los **tokens de acceso** que emite el Authorization Server suelen tener formato **JWT**. Así, OAuth2 responde a “¿cómo consigo el token de forma segura?” y JWT a “¿qué forma tiene ese token?”.

BUENAS PRÁCTICAS DE SEGURIDAD

Servir siempre sobre HTTPS; usar tiempos de expiración cortos para el `access token`; almacenar los `refresh token` de forma segura (cookies `httpOnly`); implementar rotación y revocación de tokens; y validar siempre la firma y el `scope` en cada petición.

4.5 Documentación de API

Una API sin documentación es, en la práctica, una API que nadie puede usar con confianza. La **documentación** es el contrato público que describe qué endpoints existen, qué parámetros aceptan, qué devuelven y qué errores pueden producir. Una buena documentación reduce el soporte, acelera la integración y actúa como fuente única de verdad entre el equipo que produce la API y quienes la consumen.

4.5.1 Herramientas de documentación: Swagger / OpenAPI

La **Especificación OpenAPI (OAS)**, antes conocida como **Swagger**, es el estándar de facto para describir APIs REST de forma **legible por humanos y por máquinas**. Se escribe en un archivo `YAML` o `JSON` que define, de manera estructurada, todos los aspectos de la API. A partir de ese único documento se pueden generar automáticamente páginas de documentación interactiva, clientes en distintos lenguajes y hasta esqueletos del servidor.

SWAGGER VS OPENAPI

“Swagger” fue el nombre original del proyecto. En 2016 la especificación se donó a la Linux Foundation y se renombró a **OpenAPI**. Hoy, **OpenAPI** es la **especificación**, y **Swagger** es el nombre de un conjunto de **herramientas** (Swagger UI, Swagger Editor, Swagger Codegen) que trabajan con esa especificación.

Las herramientas más habituales del ecosistema son:

Herramienta	Función
Swagger Editor	Editor en línea para escribir y validar el documento OpenAPI.
Swagger UI	Genera documentación interactiva donde se pueden probar los endpoints desde el navegador.
Swagger Codegen / OpenAPI Generator	Genera código de clientes y servidores a partir de la especificación.
Redoc	Alternativa a Swagger UI con una presentación de documentación más editorial.
Postman	Importa documentos OpenAPI para probar y automatizar peticiones.

Un fragmento mínimo de un documento OpenAPI 3.0 que describe un endpoint de listado se vería así:

```
openapi: 3.0.3
info:
  title: API de Biblioteca
  version: 1.0.0
paths:
  /api/libros:
    get:
      summary: Lista todos los libros
```

```
responses:
  '200':
    description: Operación exitosa
    content:
      application/json:
        schema:
          type: array
          items:
            $ref: '#/components/schemas/Libro'
components:
  schemas:
    Libro:
      type: object
      properties:
        id: { type: integer, example: 42 }
        titulo: { type: string, example: Clean Code }
        anio: { type: integer, example: 2008 }
```

DOCUMENTACIÓN EN LARAVEL

En el ecosistema Laravel, paquetes como [darkaonline/l5-swagger](#) o [knuckleswtf/scribe](#) permiten generar documentación OpenAPI/Swagger automáticamente a partir de anotaciones en los controladores o del análisis de las rutas, manteniendo la documentación sincronizada con el código real.

DOCUMENTACIÓN COMO CONTRATO VIVO

El enfoque **API-first** propone escribir primero el documento OpenAPI y, a partir de él, generar tanto el servidor como los clientes y las pruebas. Así, la documentación deja de ser un anexo que se escribe al final — y se desactualiza — para convertirse en el **contrato** que gobierna el desarrollo desde el inicio.

Síntesis de la unidad

A lo largo de esta unidad transformamos nuestra concepción del backend: de un generador de páginas a un **proveedor de servicios**. Establecimos que **REST** es un estilo arquitectónico que organiza la API en torno a **recursos**, manipulados mediante **métodos HTTP** y comunicados con **códigos de estado** semánticos. Vimos que el buen diseño RESTful exige consistencia, versionado y un manejo de errores estandarizado.

Conocimos **GraphQL** como alternativa que otorga al cliente control sobre la forma de los datos, con sus propios compromisos. Distinguimos **autenticación** de **autorización** y estudiamos **JWT** y **OAuth2** como piezas complementarias de la seguridad sin estado. Finalmente, comprendimos que documentar con **OpenAPI/Swagger** convierte a la API en un contrato claro y verificable. Estos fundamentos te preparan para diseñar y construir servicios web profesionales sobre Laravel en las prácticas de laboratorio.